N1: plan drift (Tier 3, item 39)

The project assumes workload cost is hardware-invariant and only machine capability changes. For a relational optimiser that assumption is false. PostgreSQL's planner reads hardware-derived settings and picks a different plan on different hardware, so the dev-side cost vector describes a plan production will not run.

Measured on ONE machine, varying only the planner's hardware *beliefs*. Hardware capability is therefore constant by construction, and every runtime change below is plan divergence with no capability confound — an isolation a two-machine experiment cannot achieve, because there both terms move together.

22 TPC-H queries x 6 environments, 264 measurements.

Q1 — How often does the plan change, and what changes it?

20 of 22 queries change plan across environments. Distinct plans per query: median 3, max 6.

Single-axis changes from the `mid` baseline, so each hardware belief is attributable:

hardware belief changed	plans changed	%	runtime ratio when plan changed	when plan unchanged
max_parallel_workers_per_gather 2 -> 8	12/22	55%	1.268x	1.123x
random_page_cost 4.0 -> 1.1	9/22	41%	1.392x	1.282x
work_mem 4MB -> 64MB	4/22	18%	2.744x	2.484x

Q2 — How much runtime variation is the plan, not the hardware?

Every pair of environments, split by whether the plan signature was identical. Pairs with the same plan isolate the execution-settings effect; pairs with different plans carry plan change plus execution.

environment pairs	n	geometric-mean runtime swing
same plan	110	**2.163x**
different plan	220	**2.494x**

Mann-Whitney U (different > same): U = 14498, p = 1.67e-03.

Roughly 16% of the cross-environment runtime variation is attributable to the plan changing rather than to the same plan running under different settings. On a machine whose hardware never changed.

Q3 — Does plan drift predict where the cross-machine model fails?

If plan volatility is a *mechanism* for CETP's error rather than a separate phenomenon, queries whose plans are unstable should be the queries predicted worst across machines.

per-query error	vs	Pearson r	p	Spearman rho	p
cmp_only	n_plans	−0.413	0.063	−0.530	0.013
cmp_only	n_dop_levels	−0.551	0.010	−0.529	0.014
analytical	n_plans	−0.446	0.043	−0.660	0.001
analytical	n_dop_levels	−0.550	0.010	−0.638	0.002

Reading Q3

Inverted and significant (r = -0.551, p = 0.010): plan-volatile queries are predicted *better*. Worth understanding before building on it — a plausible reading is that volatile queries are the parallelisable ones, and parallelism is exactly what the multi-core compute ratio captures.

Verdict: N1 should NOT be the headline as originally framed

The hypothesis was that plan divergence is a major mechanism for cross-environment error. **The measurements refute the strong form of that, in two ways.**

It is subordinate. Plan change accounts for roughly **16%** of cross-environment runtime variation. The same plan executing under different settings already swings runtime 2.16x; changing the plan on top of that takes it to 2.49x. The difference is statistically solid (p = 1.7e-03) but it is the smaller term by a wide margin — most of the variation is the settings' effect on *execution*, not on *plan choice*.

And its sign is backwards. Plan-volatile queries are predicted **better**, not worse (r = -0.551, p = 0.010). So volatility cannot be the mechanism behind the error; if anything it marks the queries that transfer most reliably. The coherent reading, consistent with everything else in this project: a query whose plan changes when core count changes is a query the planner can *parallelise*, and parallelisable queries are exactly what the multi-core compute ratio captures — the mechanism already confirmed causally at r = 0.458, p = 0.037. Plan volatility is a proxy for parallelisability.

What the result is actually worth

A literature check (`RELATED_WORK.md`) found **no formal treatment of plan divergence across hardware** — the mechanism appears only in vendor blogs, and no paper decomposes cross-environment error this way. COSTREAM (ICDE 2024), the closest cross-hardware cost model, targets streaming operator placement where no cost-based optimiser enumerates join orders, so the question cannot arise there. The gap is real and specific to relational systems.

So this stands as a **novel measurement that refutes an intuitive hypothesis**, which is a legitimate contribution and a better one than a fragile positive. Three concrete claims survive:

1. Plan drift is **pervasive**: 20/22 queries change plan under changing hardware beliefs, median 3 distinct plans.
2. **Core count is the dominant driver** (55% of queries change plan when `max_parallel_workers_per_gather` moves 2 -> 8, against 18% for `work_mem`), independently corroborating the parallelism mechanism from a completely different measurement.

3. Plan volatility is a **usable per-query signal** for which queries transfer across hardware — cheap to compute on the dev machine alone, since it needs only repeated EXPLAIN under different settings and no production access at all. That is immediately useful to the product: it tells you which queries you can trust a prediction for.

What it does **not** support is redirecting the project toward predicting the planner's choice. On this evidence that would chase the smaller term.

Limitations

- **One machine, 22 TPC-H queries.** Hardware beliefs were varied, not hardware. That is what makes the isolation clean, but it means the result demonstrates that the planner is belief-sensitive, not directly that two specific machines disagree. Confirming the latter needs the Tier 2 machine set.
- `work_mem` and `max_parallel_workers_per_gather` affect execution as well as planning, which is precisely why Q2 splits pairs by plan signature rather than comparing settings directly.
- Plans were not **forced** across environments (`pg_hint_plan` is not installed), so the decomposition is observational: it compares pairs that happened to agree on a plan against pairs that did not. Forcing would let the same plan be run under every belief set and give a direct causal decomposition. That is the obvious next step and needs only the extension installed.
- q16 and q20 are APPROXIMATE query reconstructions (`scripts/collection/queries/README.md`).